

Dokumentacja projektu

Calisia Banking App

Aplikacja bankowa z panelem klienta, produktami finansowymi
i modelem CQRS

Data raportu: 06.06.2026

Autorzy:

Kubryn Jeremiasz

Łaszuk Jakub

Makarevich Ksenia

Nuckowski Michał

Ruczyńska Nikola

Wołczko Jakub

Olszewski Dawid

Spis treści

1	Cel i zakres aplikacji	2
2	Najważniejsze funkcje systemu	2
3	Architektura	2
3.1	Warstwy backendu	3
4	Przypadki użycia	4
5	Model domenowy	5
5.1	Wybrane reguły domenowe	7
6	Schemat bazy danych	7
6.1	CalisiaWriteDb	9
6.2	CalisiaReadDb	9
7	API REST	9
8	Frontend	10
8.1	Ekrany aplikacji	10
9	Przepływy procesów	16
9.1	Rejestracja i logowanie	16
9.2	Przelew i dashboard	16
9.3	Kredyt i operacje kartowe	17
10	Mechanizm CQRS i Outbox	17
11	Bezpieczeństwo	18
12	Testy i narzędzia	18
13	Wnioski	18

1 Cel i zakres aplikacji

Calisia Banking App jest aplikacją bankową przeznaczoną do obsługi podstawowych czynności klienta banku. System umożliwia rejestrację i logowanie klienta, prezentację dashboardu finansowego, zarządzanie produktami bankowymi, wykonywanie przelewów, obsługę kart oraz kredytu gotówkowego.

Aplikacja składa się z dwóch głównych części:

- frontendu typu SPA zbudowanego w React i TypeScript,
- backendu REST API zbudowanego w .NET, podzielonego na warstwy API, Application, Domain, Infrastructure oraz ReadModel.

System korzysta z dwóch baz danych SQL Server. Baza zapisu **CalisiaWriteDb** przechowuje model domenowy oraz outbox, natomiast baza odczytu **CalisiaReadDb** przechowuje zoptymalizowane dane dashboardu.

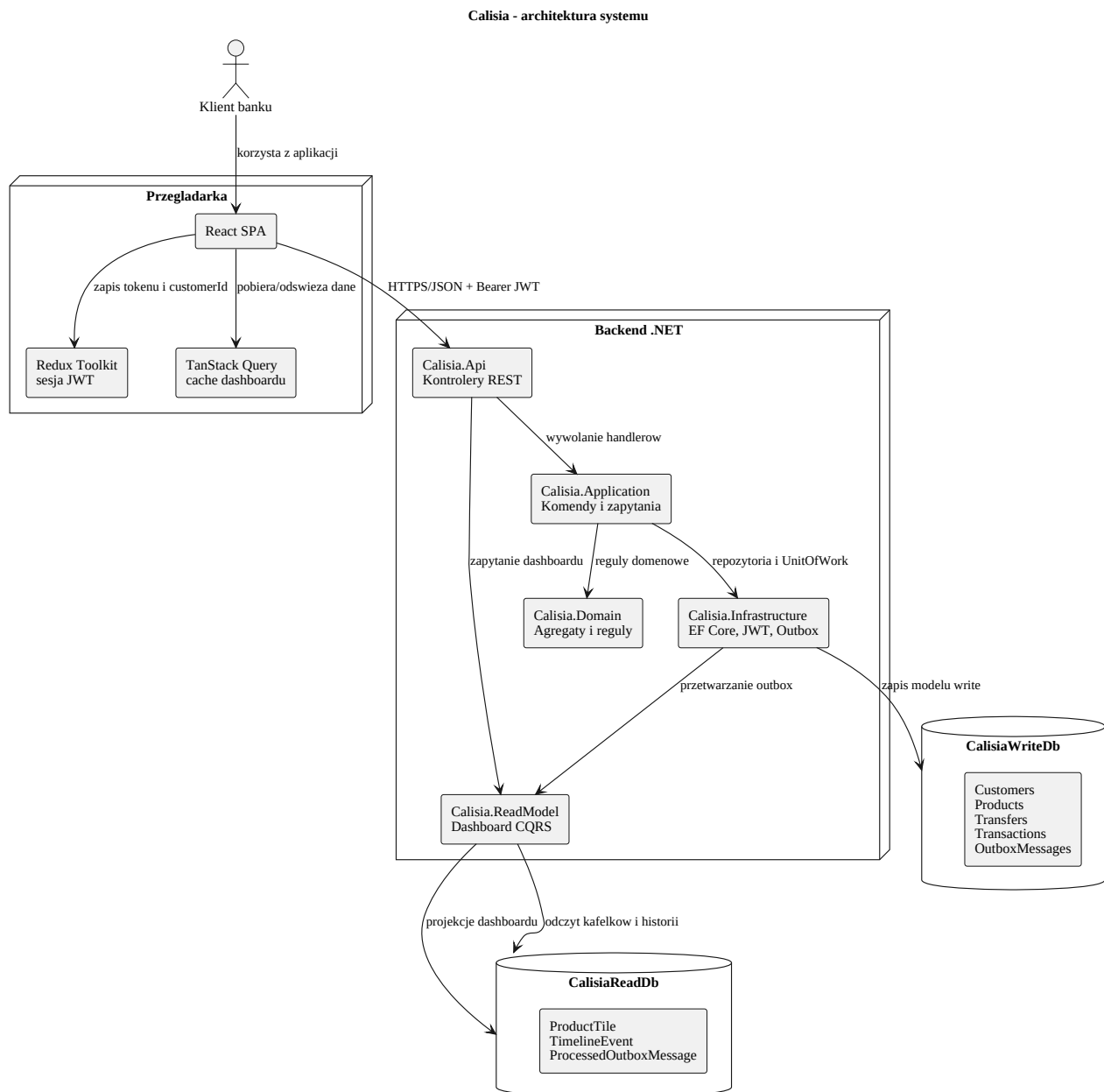
2 Najważniejsze funkcje systemu

Obszar	Opis funkcjonalności
Rejestracja klienta	Utworzenie konta klienta, zahashowanie hasła oraz automatyczne dodanie produktów startowych: konta głównego i karty debetowej. Dla klienta prestiżowego tworzona jest także karta kredytowa.
Logowanie	Weryfikacja danych dostępowych i wydanie tokenu JWT używanego przez frontend do autoryzowanych żądań.
Dashboard	Prezentacja salda, listy produktów i historii zdarzeń finansowych. Dane pochodzą z modelu odczytu.
Produkty bankowe	Obsługa kont, kart oraz kredytów. Produkty są przechowywane w jednej tabeli Products z rozróżnieniem typu przez kolumnę Discriminator .
Przelewy	Obsługa przelewów własnych oraz zewnętrznych. Przelew własny aktualizuje saldo źródła i celu, a przelew zewnętrzny tylko obciąża konto źródłowe.
Kredyt gotówkowy	Dodanie produktu typu kredyt gotówkowy zwiększa saldo konta głównego. Wcześniejsza spłata pobiera kwotę kredytu z konta głównego i usuwa produkt kredytu.
Karty	Symulacja płatności kartą debetową pobiera środki z konta głównego. Symulacja płatności kartą kredytową zmniejsza dostępne saldo karty. Spłata karty kredytowej pobiera z konta głównego wykorzystany limit.

3 Architektura

Backend stosuje podział zbliżony do Clean Architecture. Warstwa API odpowiada za kontrolery HTTP i autoryzację, Application za przypadki użycia, Domain za reguły biznesowe,

Infrastructure za dostęp do bazy, JWT i outbox, a ReadModel za projekcje dashboardu.



Rysunek 1: Architektura systemu Calisia

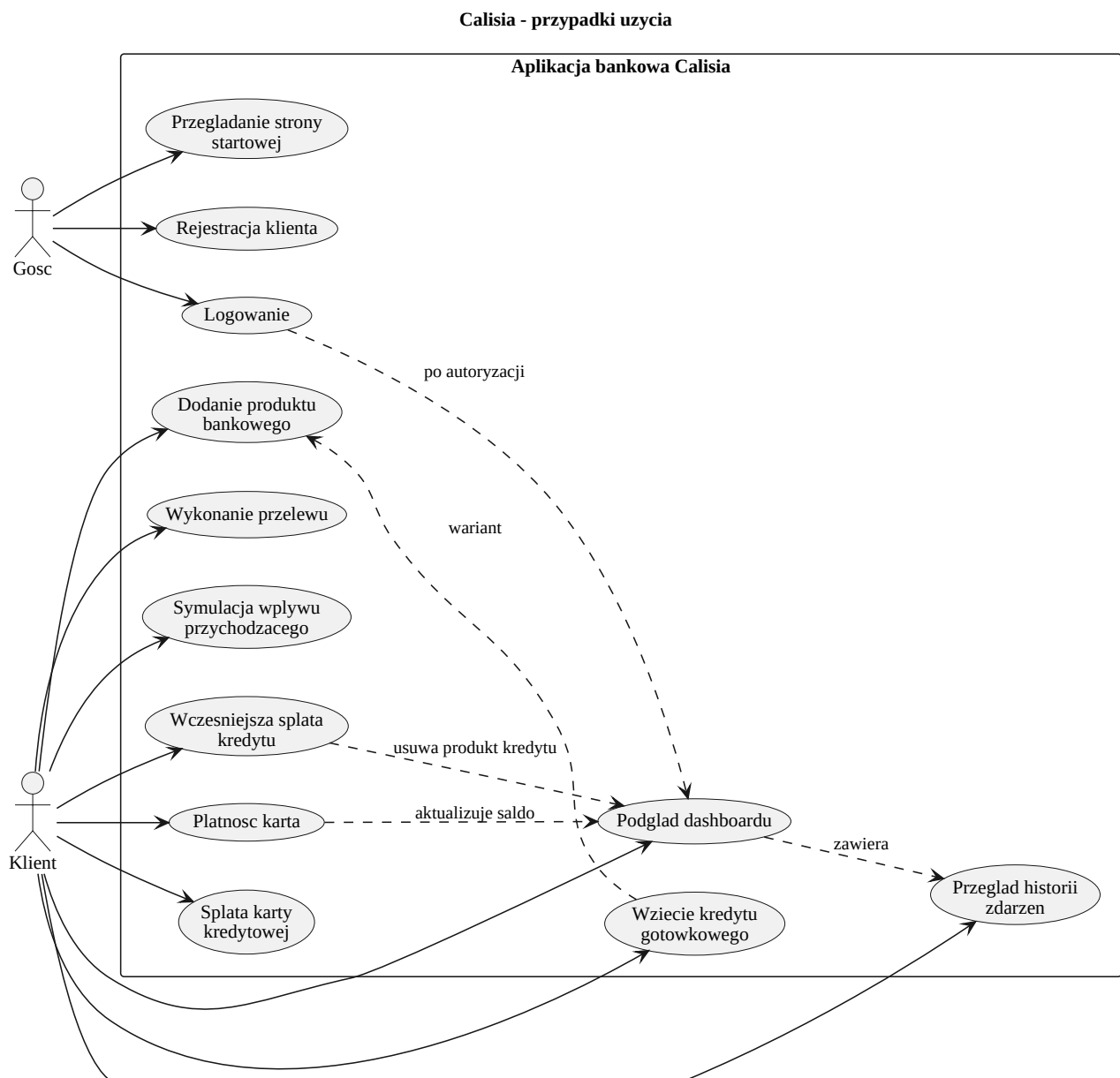
3.1 Warstwy backendu

Warstwa	Odpowiedzialność
Calisia.Api	Kontrolery REST, middleware obsługi wyjątków, konfiguracja CORS, JWT i Swagger/OpenAPI.
Calisia.Contracts	Kontrakty żądań i odpowiedzi HTTP, między innymi rejestracja, logowanie, produkty, karty i przelewy.
Calisia.Application	Komendy i zapytania aplikacyjne, walidacja przypadków użycia oraz koordynacja repozytoriów i jednostki pracy.
Calisia.Domain	Agregaty, encje, value objects, enumy i zdarzenia domenowe.

Warstwa	Odpowiedzialność
Calisia.Infrastructure	Implementacje repozytoriów EF Core, konfiguracje mapowania, hashowanie haseł, tokeny JWT oraz procesor outbox.
Calisia.ReadModel	Odczyt dashboardu, modele ProductTile i TimelineEvent oraz projekcje zdarzeń domenowych.

4 Przypadki użycia

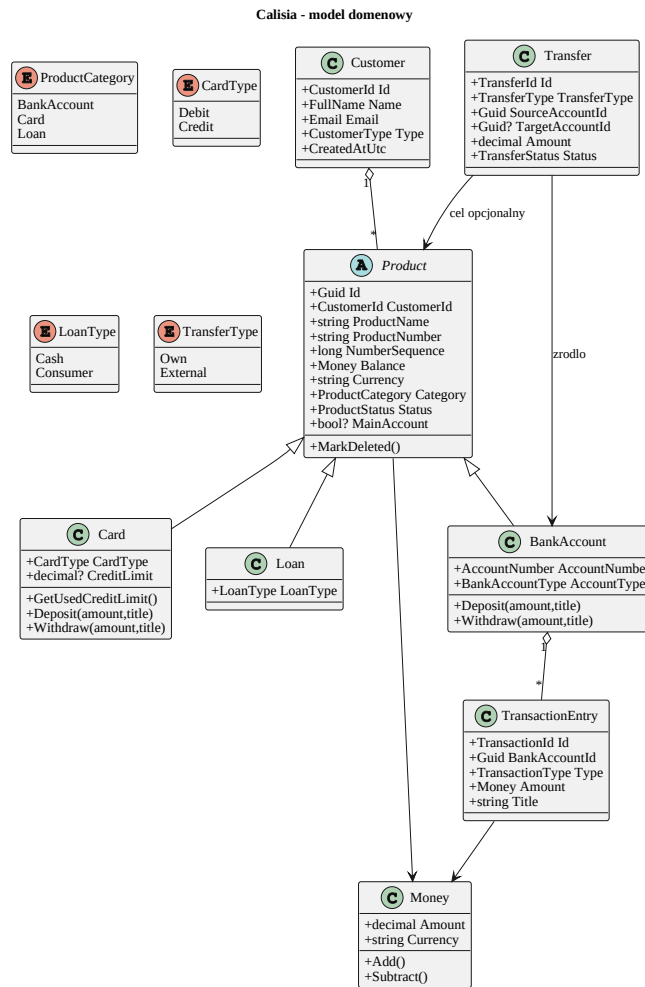
Diagram przypadków użycia pokazuje czynności dostępne dla gościa oraz zalogowanego klienta.



Rysunek 2: Przypadki użycia aplikacji

5 Model domenowy

Centralnym elementem domeny jest abstrakcyjny produkt bankowy **Product**. Dziedziczą po nim: **BankAccount**, **Card** i **Loan**. Takie podejście pozwala przechowywać różne produkty finansowe w jednej tabeli, przy zachowaniu osobnych reguł biznesowych dla kont, kart i kredytów.



Rysunek 3: Model domenowy

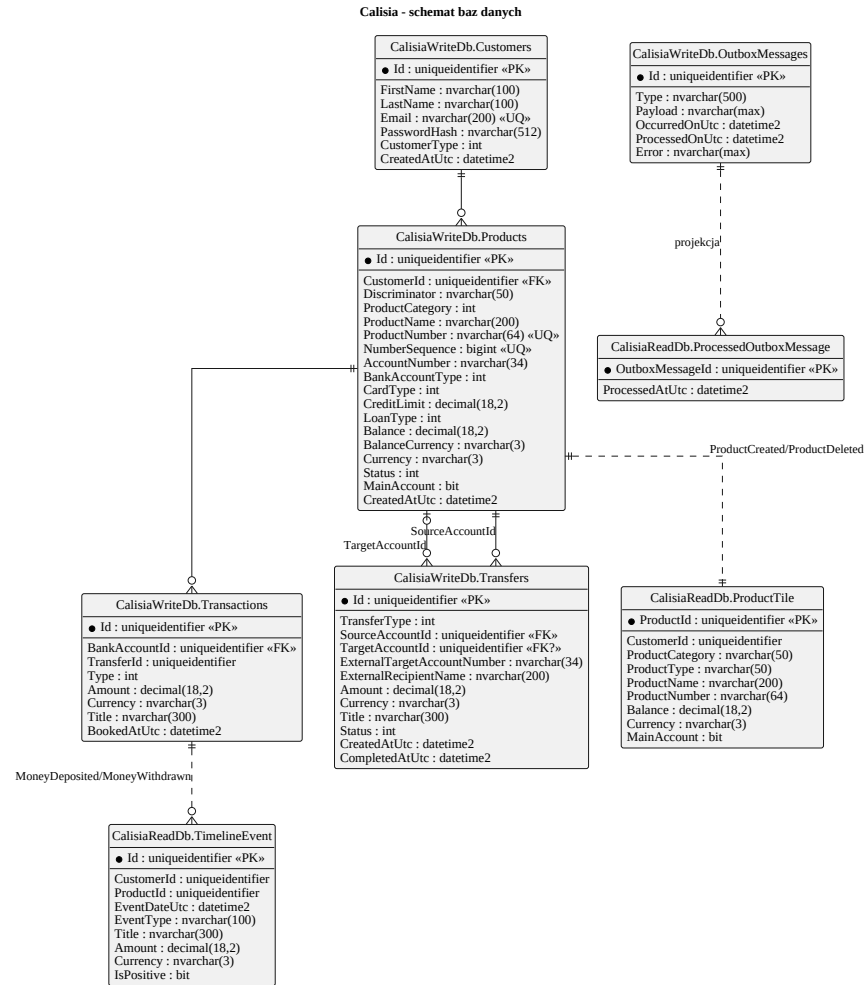
5.1 Wybrane reguły domenowe

- Konto bankowe może przyjmować wpłaty i wypłaty, a każda operacja generuje wpis transakcji oraz zdarzenie domenowe.
- Karta debetowa nie posiada limitu kredytowego i przy płatności obciąża konto główne klienta.
- Karta kredytowa posiada `CreditLimit`; jej saldo reprezentuje dostępny limit. Płatność zmniejsza saldo karty, a spłata zwiększa je do wysokości limitu.
- Kredyt gotówkowy jest produktem typu `Loan`. Wzięcie kredytu zasila konto główne, natomiast wcześniejsza spłata pobiera kwotę kredytu z konta głównego i usuwa produkt kredytu.
- Przelew własny aktualizuje dwa produkty: źródłowy i docelowy. Przelew zewnętrzny aktualizuje tylko produkt źródłowy.

6 Schemat bazy danych

System korzysta z dwóch baz: zapisu i odczytu. Baza zapisu przechowuje pełny stan domenowy, a baza odczytu przechowuje uproszczony widok potrzebny dashboardowi.

∞



Rysunek 4: Schemat baz danych write/read

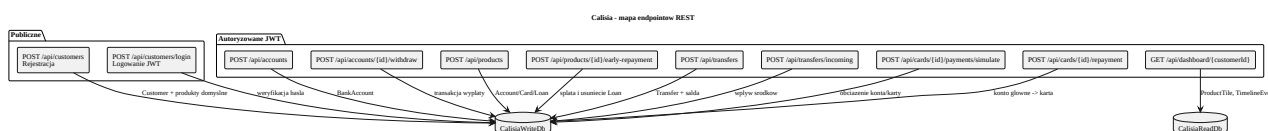
6.1 CalisiaWriteDb

Tabela	Znaczenie
Customers	Dane klienta, email, hash hasła, typ klienta oraz data utworzenia.
Products	Wspólna tabela produktów bankowych. Kolumna Discriminator rozróżnia konto, kartę i kredyt.
Transactions	Historia operacji na kontach bankowych.
Transfers	Dane przelewów własnych i zewnętrznych.
OutboxMessages	Zdarzenia domenowe oczekujące na projekcję do bazy odczytu.

6.2 CalisiaReadDb

Tabela	Znaczenie
ProductTile	Kafelki produktów widoczne na dashboardzie: typ, numer, saldo, waluta i oznaczenie konta głównego.
TimelineEvent	Historia zdarzeń finansowych prezentowana w panelu użytkownika.
ProcessedOutboxMess	Rejestr przetworzonych wiadomości outbox, chroniący przed podwójną projekcją.

7 API REST



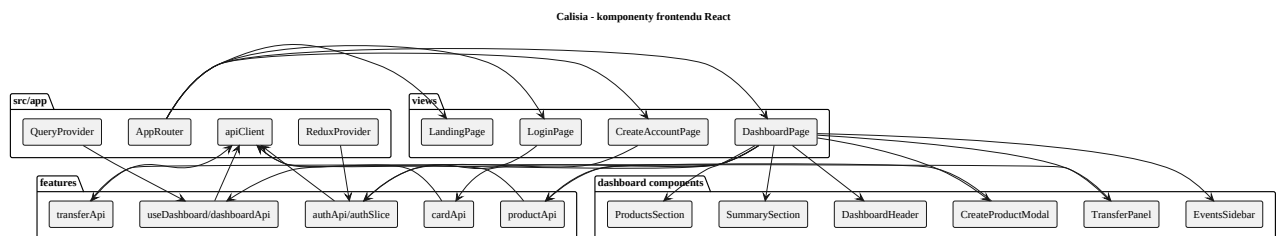
Rysunek 5: Mapa endpointów API

Endpoint	Opis
POST /api/customers	Rejestracja klienta.
POST /api/customers/login	Logowanie i wydanie JWT.
GET /api/dashboard/{customerId}	Pobranie danych dashboardu z modelu odczytu.
POST /api/accounts	Otwarcie nowego konta bankowego.
POST /api/accounts/{id}/withdraw	Wypłata środków z konta.
POST /api/products	Dodanie produktu bankowego: konto, karta lub kredyt.
POST /api/products/{id}/early-repayment	Wcześniejsza spłata kredytu gotówkowego.

Endpoint	Opis
POST /api/transfers	Utworzenie przelewu własnego lub zewnętrznego.
POST /api/transfers/incoming	Symulacja wpływu przychodzącego.
POST /api/cards/{id}/payments/	Symulacja płatności kartą debetową lub kredytową.
POST /api/cards/{id}/repayment	Splata wykorzystanego limitu karty kredytowej.

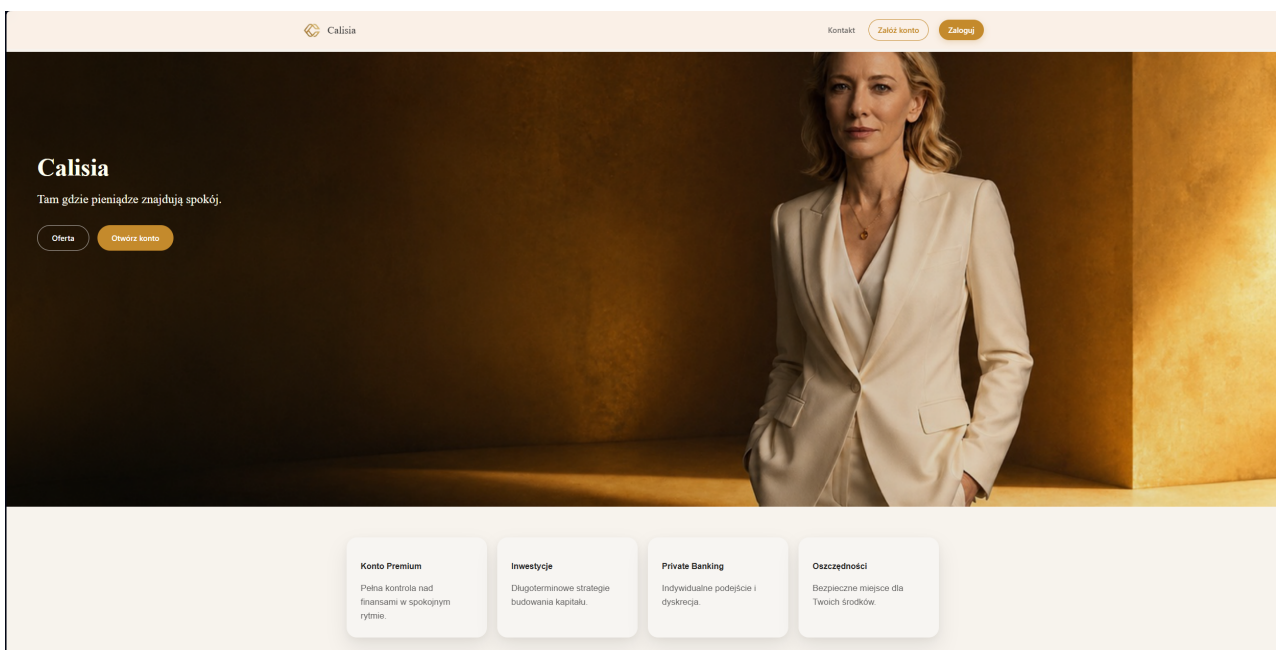
8 Frontend

Frontend jest aplikacją React SPA. Routing realizuje **AppRouter**, stan sesji przechowywany jest w Redux Toolkit, a pobieranie i odświeżanie danych dashboardu obsługuje TanStack Query.

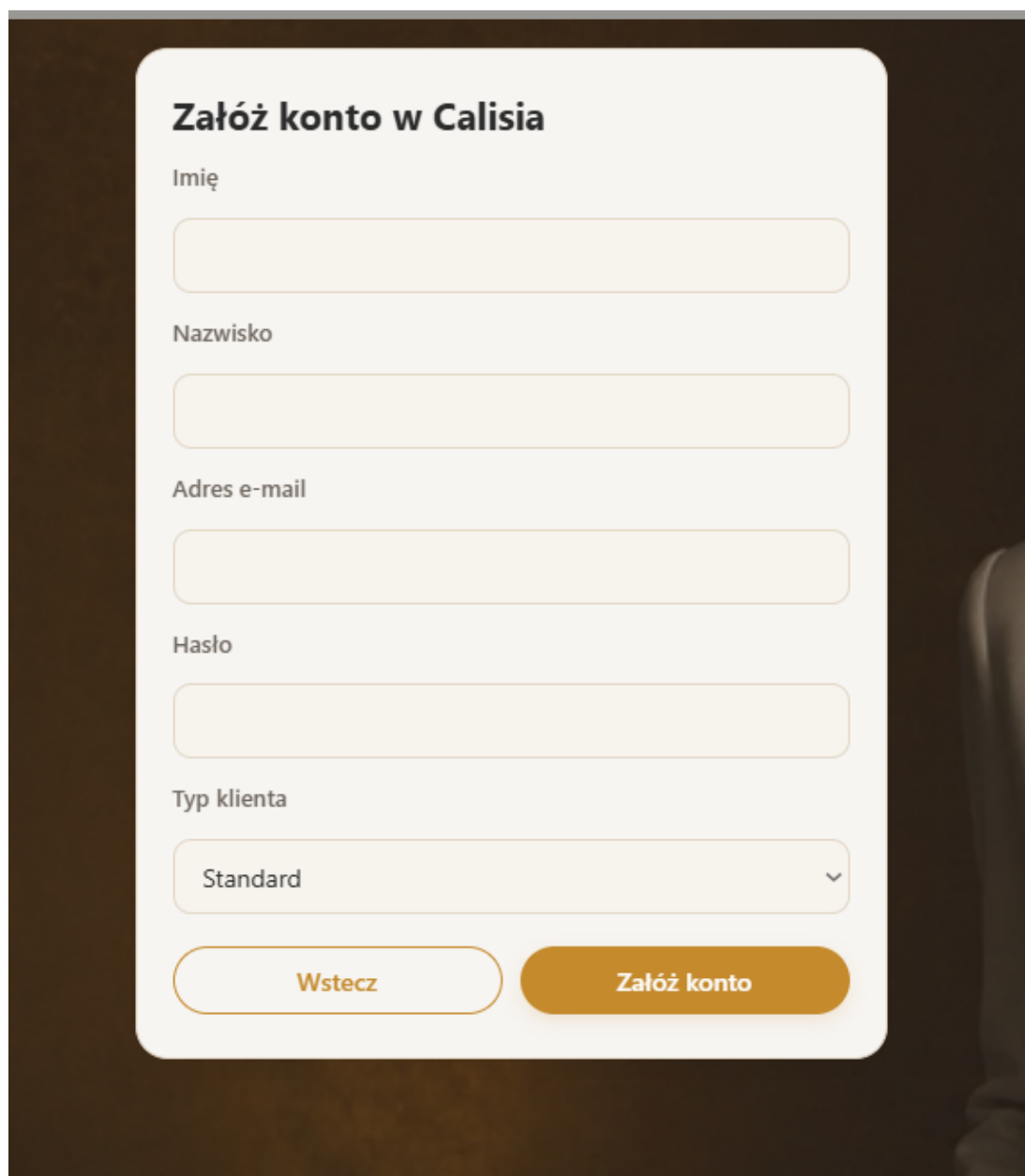


Rysunek 6: Komponenty frontendu

8.1 Ekran aplikacji



Rysunek 7: Strona startowa aplikacji



Załącz konto w Calisia

Imię

Nazwisko

Adres e-mail

Hasło

Typ klienta

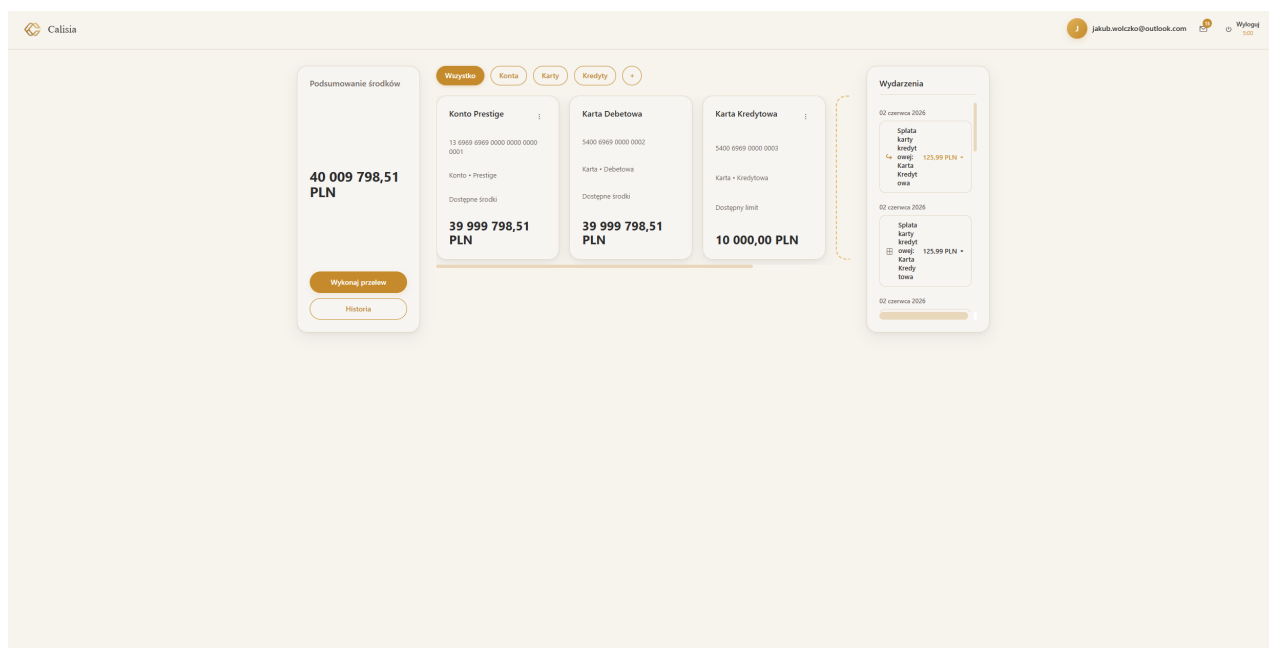
Standard

Wstecz

Załącz konto

The image shows a registration form titled 'Załącz konto w Calisia' (Create account in Calisia). It is set against a dark brown background. The form itself is a light cream color with rounded corners. It contains five input fields: 'Imię' (First name), 'Nazwisko' (Last name), 'Adres e-mail' (Email address), and 'Hasło' (Password). Below these is a dropdown menu for 'Typ klienta' (Client type), which currently shows 'Standard'. At the bottom of the form are two buttons: a light cream button with a brown border labeled 'Wstecz' (Back) and a solid brown button labeled 'Załącz konto' (Create account).

Rysunek 8: Formularz rejestracji klienta



Rysunek 9: Dashboard klienta

Karta • Debetowa

Do przelewów

Typ przelewu

Zewnętrzny

Konto źródłowe

Konto Prestige • 13 6969 6969 0000 0000 0001

Konto docelowe

Odbiorca

Tytuł przelewu

Kwota

Anuluj Wykonaj przelew

Rysunek 10: Panel tworzenia przelewu

Otwórz nowy produkt

Konto

Karta

Kredyt

Wnioskowana kwota (PLN)

0.00

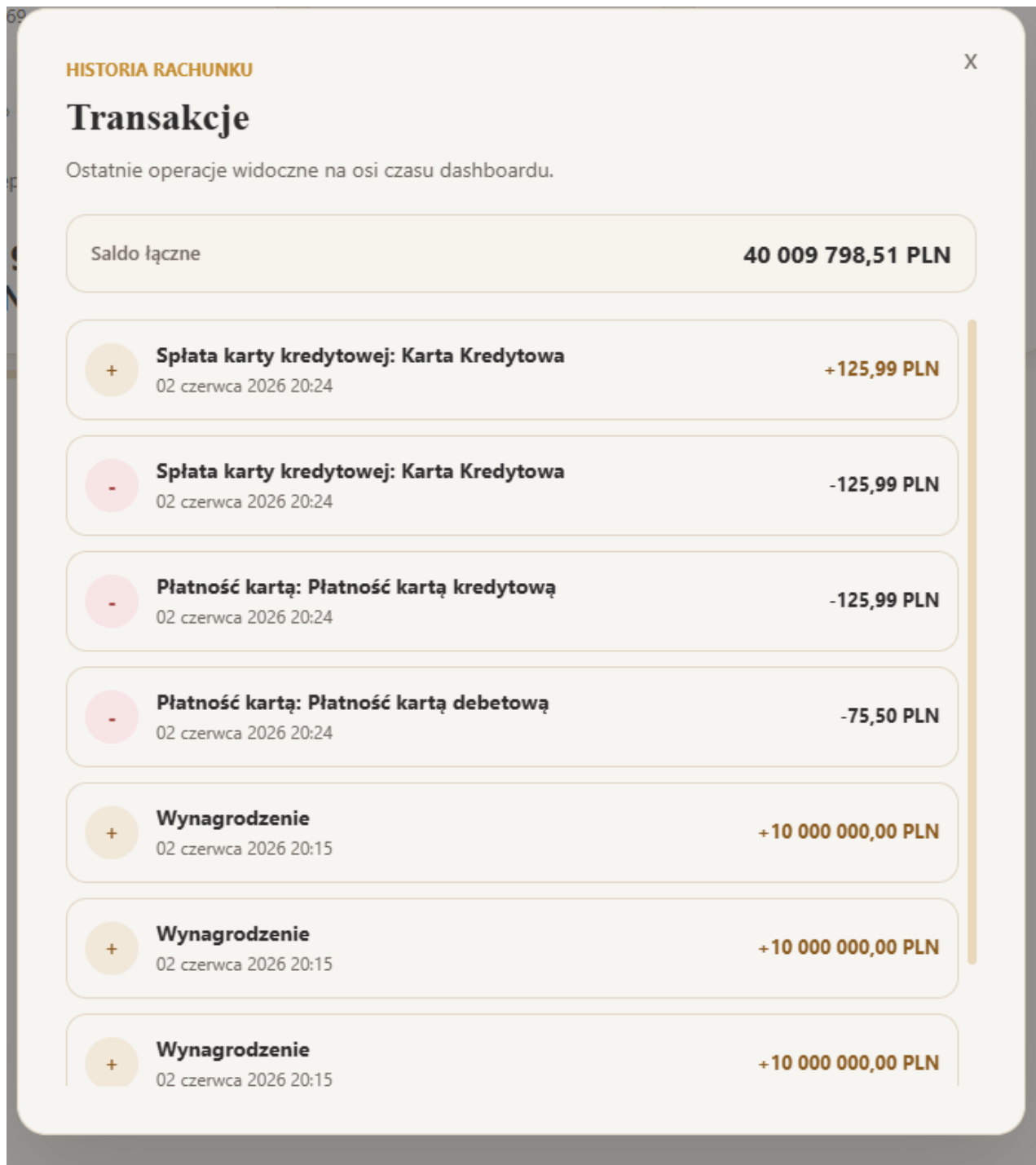
PLN

Nazwa własna (opcjonalnie)

np. Kredyt Gotówkowy

Zatwierdź

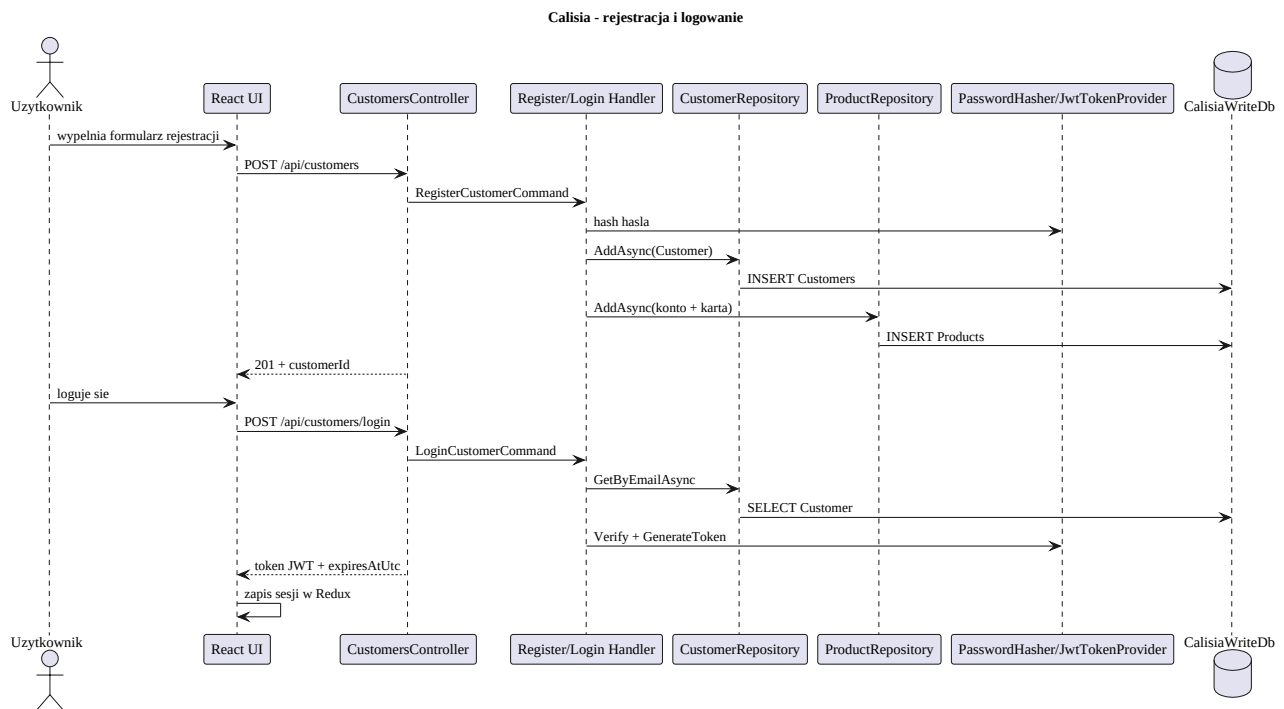
Rysunek 11: Dodanie produktu typu kredyt gotówkowy



Rysunek 12: Historia transakcji i zdarzeń

9 Przepływy procesów

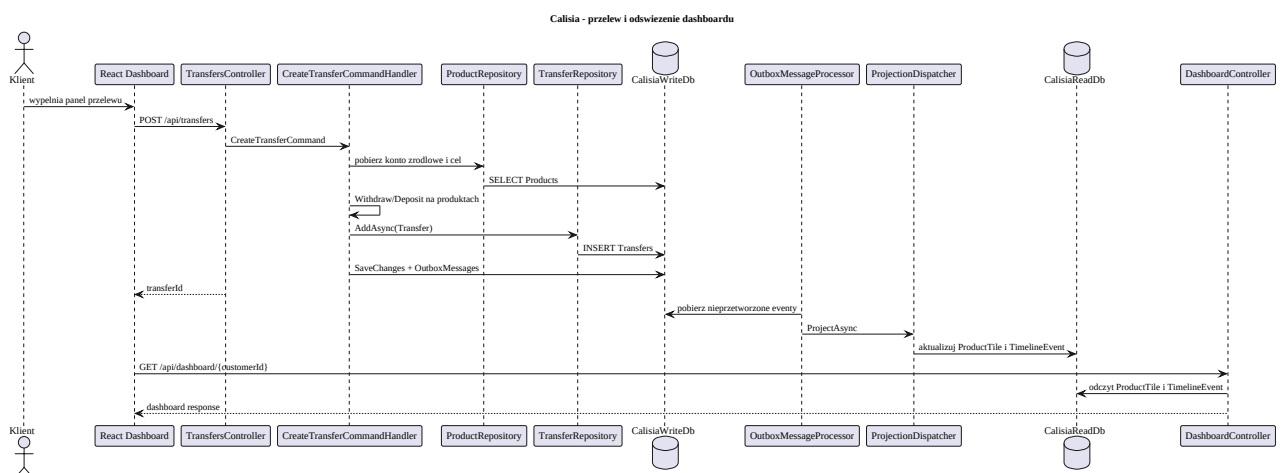
9.1 Rejestracja i logowanie



Rysunek 13: Sekwencja rejestracji i logowania

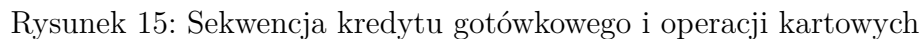
Podczas rejestracji aplikacja tworzy klienta i produkty startowe. Podczas logowania backend weryfikuje hash hasła i zwraca token JWT. Token jest następnie dołączany przez frontend do żądań autoryzowanych.

9.2 Przelew i dashboard



Rysunek 14: Sekwencja przelewu i odświeżenia dashboardu

9.3 Kredyt i operacje kartowe



10 Mechanizm CQRS i Outbox

Takie rozwiązanie pozwala:

- 17

11 Bezpieczeństwo

Autoryzacja użytkownika oparta jest o token JWT. Po zalogowaniu frontend zapisuje token i przekazuje go w nagłówku `Authorization: Bearer ....` Endpoint dashboardu dodatkowo sprawdza, czy identyfikator klienta w adresie odpowiada klientowi z tokenu.

Hasła nie są przechowywane jawnie. Backend używa hashera PBKDF2, a w bazie przechowywany jest wynik hashowania. Middleware obsługi wyjątków mapuje błędy domenowe i aplikacyjne na odpowiedzi HTTP.

12 Testy i narzędzia

Projekt zawiera testy jednostkowe oraz integracyjne backendu. Testy jednostkowe pokrywają między innymi:

- tworzenie produktów bankowych,
- kredyt gotówkowy i jego spłatę,
- płatności kartą debetową i kredytową,
- spłatę karty kredytowej,
- przelewy,
- hashowanie haseł i tokeny JWT,
- projekcje modelu odczytu.

Do ręcznego testowania endpointów przygotowano kolekcję Bruno. Zawiera ona żądania dla klientów, dashboardu, produktów, przelewów oraz kart.

13 Wnioski

Calisia Banking App jest aplikacją demonstracyjną pokazującą bankowość elektroniczną z wyraźnym podziałem warstw, modelem domenowym oraz odseparowanym modelem odczytu. Najważniejszą cechą architektury jest połączenie DDD, CQRS i outbox, które umożliwia spójne przetwarzanie zdarzeń finansowych oraz szybkie prezentowanie danych dashboardu.

Frontend zapewnia użytkownikowi prosty przepływ od rejestracji i logowania po codzienną obsługę rachunku, produktów, kart, kredytu oraz historii operacji. Backend koncentruje reguły biznesowe w domenie i udostępnia je przez czytelne endpointy REST.